

Estudio e implementación paralela de métodos explícitos multipaso

RESOLUCIÓN DE ECUACIONES DE ADVECCIÓN-REACCIÓN-DIFUSIÓN

- ▶ Introducción
 - ▶ Métodos numéricos y EDOs
 - ▶ Métodos de Runge-Kutta
 - ▶ Métodos de Adams-Bashforth
 - ▶ Métodos de Adams-Bashforth-Moulton
- ▶ Tecnologías de Paralelismo
- ▶ Implementación multihebra
- ▶ Implementación para GPU
- ▶ Conclusiones

Introducción

Objetivos

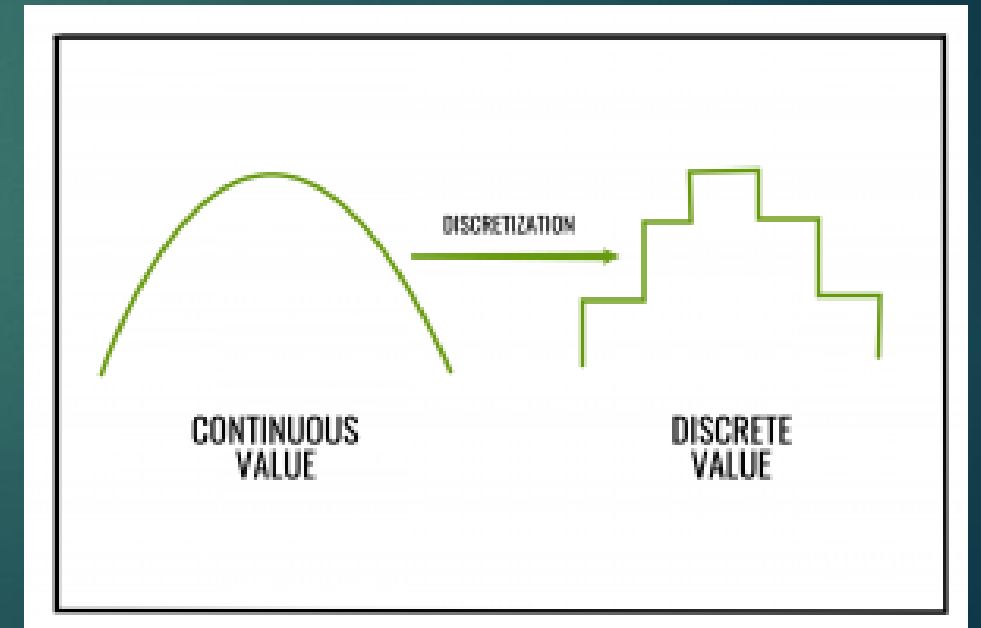
4

- ▶ Adquirir una **base sólida** de los conceptos
- ▶ Estudio de las **tecnologías de paralelización**
- ▶ **Implementación** eficiente
- ▶ Realización de **experimentos exhaustivos**
- ▶ **Análisis crítico** de los resultados

Métodos numéricos y EDOs

5

- ▶ Métodos numéricos: aproximar soluciones
- ▶ Ecuaciones diferenciales: **Función $f(x)$ + variable t + derivada dy/dt**
- ▶ Ecuaciones en derivadas parciales: Varias variables
- ▶ Ecuaciones de **Advección-Reacción-Difusión** (ARD)
- ▶ Ecuaciones diferenciales ordinarias (EDOs): sólo 1 variable
- ▶ Problemas de valor inicial:
PVI: $\frac{dy}{dt} = f(t_n, y_n), \quad y(t_0) = y_0 \in \mathbb{R}^d, \quad t \in [t_0, t_f]$
- ▶ Proceso:
Pasar de $y(t_0)$ a $y(t_f)$ iterativamente
- ▶ Discretización del dominio
de t en N puntos separados Δt



Discretización ([geeksforgeeks](https://www.geeksforgeeks.org/))

Tipos de métodos numéricos

6

- ▶ ¿Cómo obtenemos $y(t_n + \Delta t)$ a partir de $y(t_n)$?

- ▶ **Expresión general:**

$$y_{n+1} = y_n + \Phi(t_n, y_n, y_{n+1}, \dots, y_{n-k} \Delta t)$$

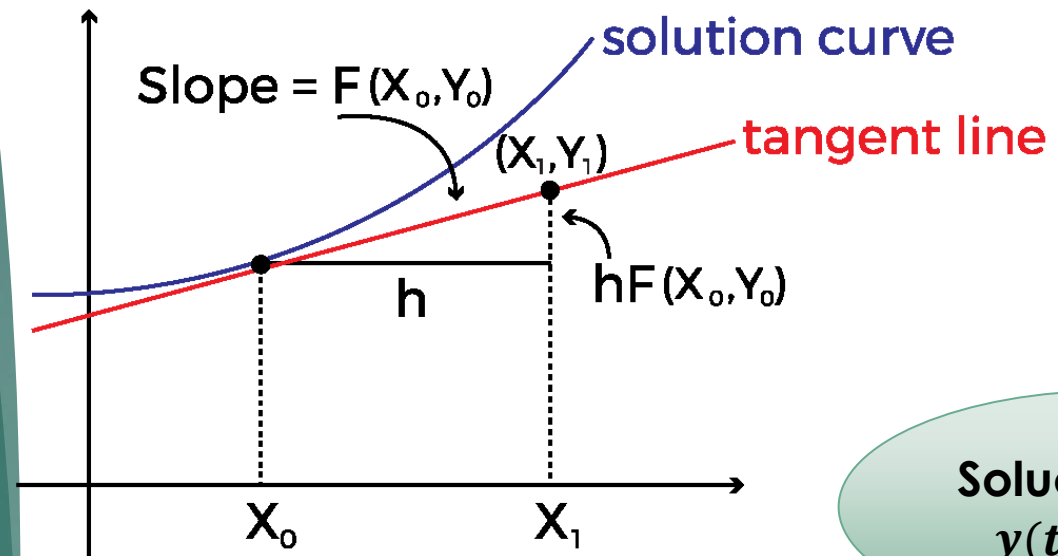
- ▶ Métodos explícitos: **no** se utiliza $f(t_{n+1}, y_{n+1})$
- ▶ Métodos implícitos: **sí** se utiliza $f(t_{n+1}, y_{n+1})$
- ▶ Métodos de **un solo paso**: sólo se utiliza $f(t_n, y_n)$
- ▶ Métodos **multipaso**: se utilizan $f(t_{n-1}, y_{n-1})$, $f(t_{n-2}, y_{n-2})$, ...

Método de Euler

- Explícito y de un paso
- $y_{n+1} = y_n + \Delta t \cdot f(t_n, y_n)$
- Calculamos cada paso **iterativamente** apoyándonos en el anterior, desde t_0 hasta t_f

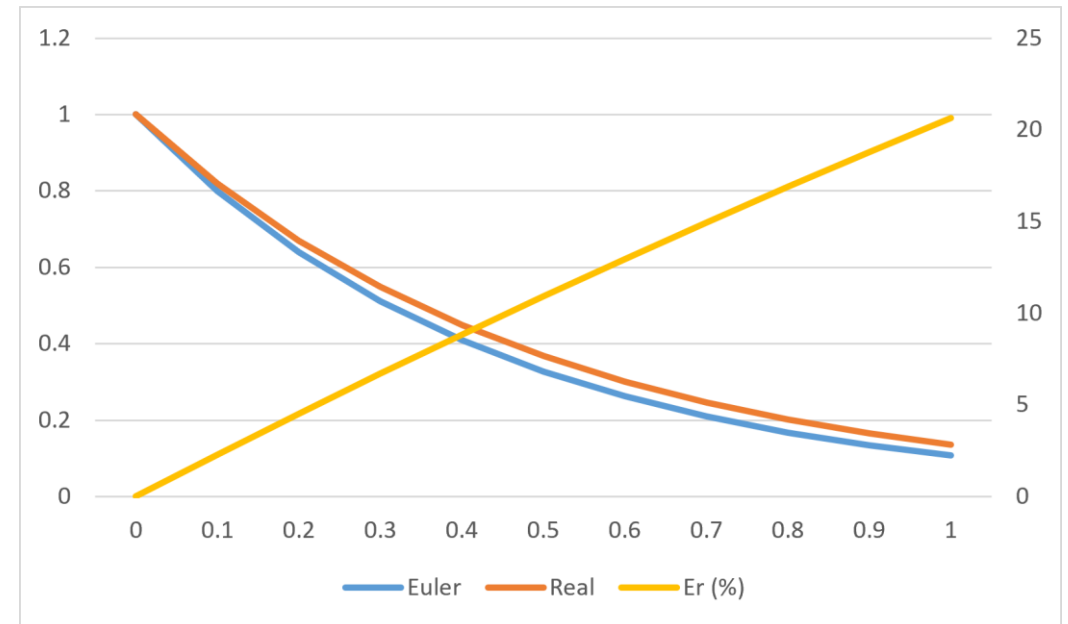
- **Ejemplo:**

- Derivada: $\frac{dy}{dt} = -2y$
- Valor Inicial: $y(t_0) = y(0) = 1$
- Dominio: $[t_0, t_f] = [0.0, 1.0]$
- Tamaño de paso: $\Delta t = 0.1$



Aplicación del método de Euler ([calcworkshop](http://calcworkshop.com))

Solución real:
 $y(t) = e^{-2t}$



Comparativa entre Euler y solución real (elaboración propia)

Métodos de Runge-Kutta

- ▶ Órdenes 1, 2, 3, y 4
- ▶ Partiendo de un PVI, calcularemos:

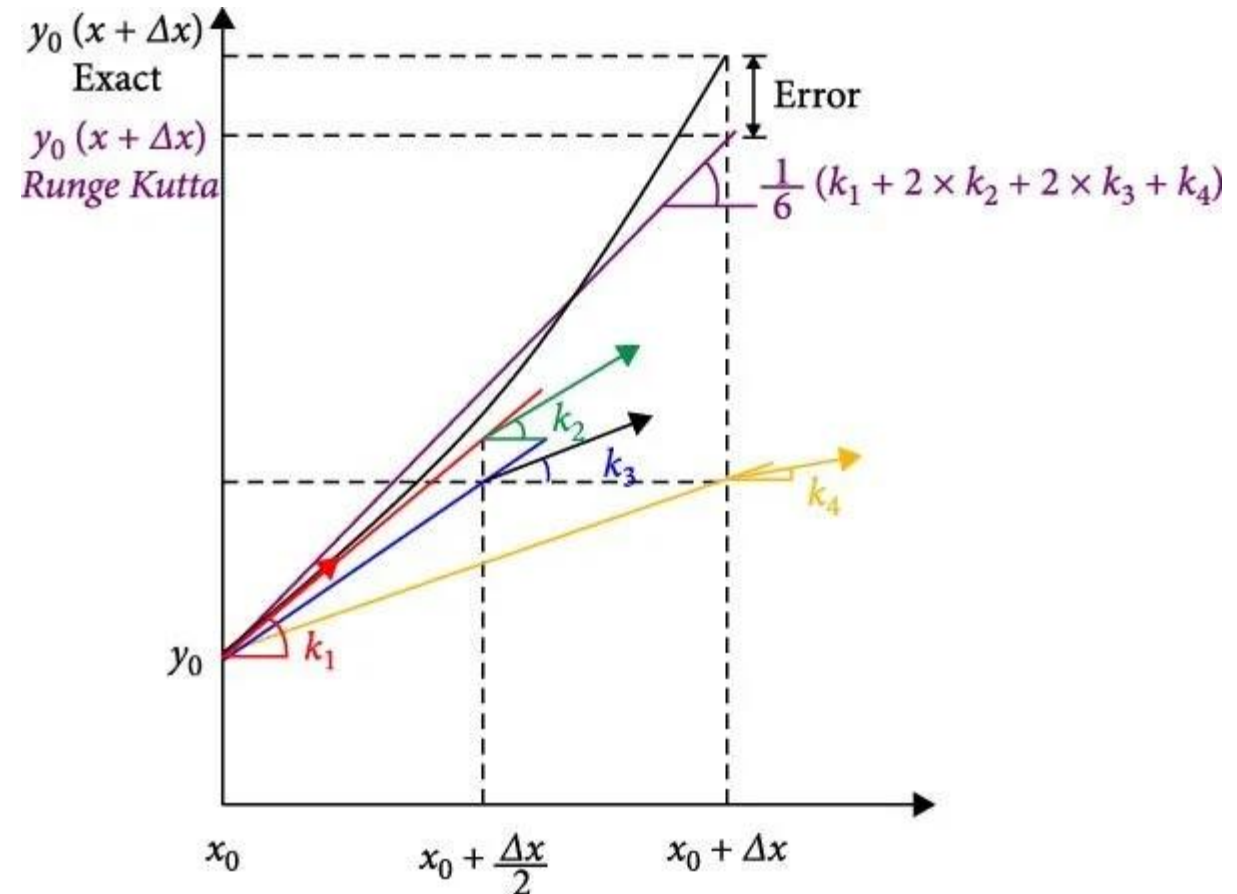
$$y_{n+1} = y_n + \Delta t/6 \cdot (K_1 + 2K_2 + 2K_3 + K_4)$$

$$K_1 = f(t_n, y_n)$$

$$K_2 = f(t_n + \Delta t/2, y_n + K_1 \cdot \Delta t/2)$$

$$K_3 = f(t_n + \Delta t/2, y_n + K_2 \cdot \Delta t/2)$$

$$K_4 = f(t_n + \Delta t, y_n + K_3 \cdot \Delta t)$$



Método de Runge-Kutta ([onlinelibrary](https://www.onlinelibrary.com))

Métodos de Adams-Bashforth

- ▶ Métodos multipaso explícitos
- ▶ Órdenes 1, 2, 3, y 4
- ▶ Se necesitan valores de arranque, los conseguiremos con RK4
- ▶ Cálculo iterativo con Δt

Algoritmo 2 Adams-Bashforth general de orden k

```

for i=1; i < k; ++i do
     $y_i \leftarrow \text{arranqueRK4}(y_{i-1})$ 
end for
for i=0; i < k; ++i do
     $f_i \leftarrow \text{feval}(f_i)$ 
end for
for  $t_n = t_0 + (k-1) \cdot \Delta t$ ;  $t_n < t_f$ ;  $t_n += \Delta t$  do
     $y_k \leftarrow \text{AdamsBashforth}(k, y_{k-1}, f_n, f_{n-1}, \dots, f_{n-k})$ 
     $f_k \leftarrow \text{feval}(Y_k)$ 
    swap( $y_k, y_{k-1}$ )
    for i=1; i ≤ k; ++i do
        swap( $f_n, f_{n-1}$ )
    end for
end for
 $Y_f \leftarrow Y_{k-1}$ 

```

- ▶ Fórmula para aplicar orden 4 $\longrightarrow y_{n+1} = y_n + h/24 \cdot (55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3})$

Métodos de Adams-Bashforth-Moulton

- Sistema Predictor-Corrector
- Combina Adams-Bashforth y Adams-Moulton
- Órdenes 0, 1, 2, 3, y 4
- Se necesitan valores de arranque, los conseguiremos con RK4
- Cálculo iterativo con Δt

Algoritmo 3 Adams-Bashforth-Moulton de orden k

```

for  $i=1; i < k; ++i$  do
     $y_i \leftarrow \text{arranqueRK4}(y_{i-1})$ 
end for
for  $i=0; i < k; ++i$  do
     $f_i \leftarrow \text{feval}(y_i)$ 
end for
for  $t_n=t_0 + (k-1) \cdot \Delta t; t_n < t_f; t_n+=\Delta t$  do
     $y_{\text{pred}} \leftarrow \text{AdamsBashforth}(k, y_{k-1}, f_n, f_{n-1}, \dots, f_{n-k})$ 
     $f_{\text{pred}} \leftarrow \text{feval}(t_{n+h}, y_{\text{pred}})$ 
     $y_{\text{corr}} \leftarrow \text{AdamsMoulton}(k, y_{k-1}, f_{\text{pred}}, f_n, f_{n-1}, \dots, f_{n-k})$ 
     $f_{\text{corr}} \leftarrow \text{feval}(t_{n+h}, y_{\text{corr}})$ 
     $y_k \leftarrow \text{AdamsMoulton}(k, y_{k-1}, f_{\text{corr}}, f_n, f_{n-1}, \dots, f_{n-k})$ 
     $f_k \leftarrow \text{feval}(t_{n+h}, y_k)$ 
     $\text{swap}(y_k, y_{k-1})$ 
    for  $i=1; i \leq k; ++i$  do
         $\text{swap}(f_n, f_{n-1})$ 
    end for
end for
 $Y_f \leftarrow Y_{k-1}$ 

```

- Fórmula para aplicar orden 4  $y_{n+1} = y_n + \frac{\Delta t}{720} \cdot (251f_{n+1} + 646f_n - 264f_{n-1} + 106f_{n-2} + 19f_{n-3})$

Problemas de prueba

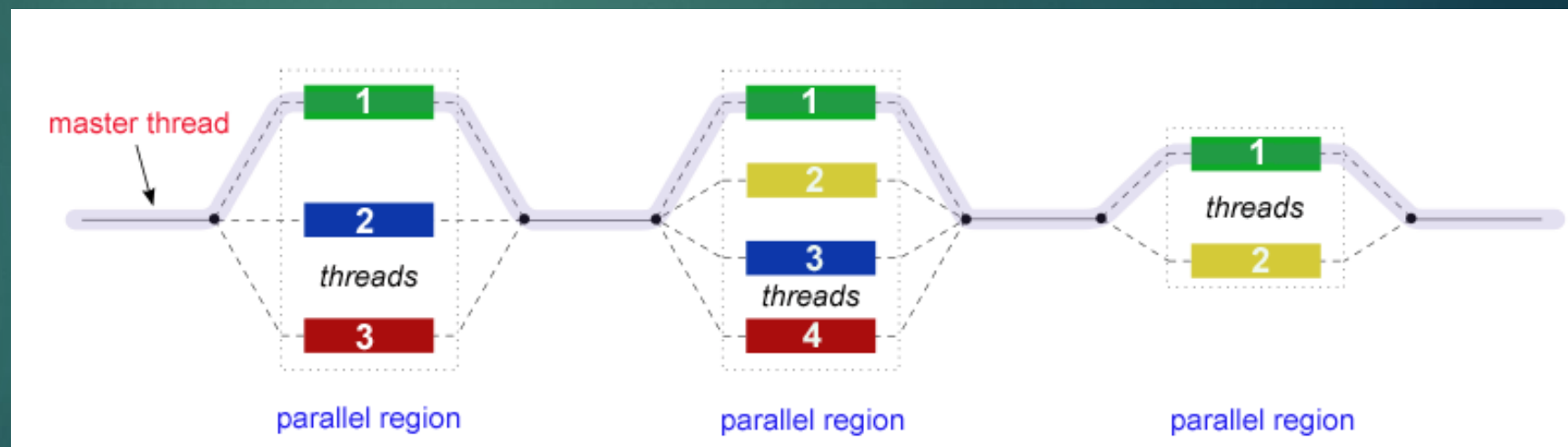
11

- ▶ **SimpleAdvDiff**: Modelo de Advección-Difusión
- ▶ **AdvDiff1D**: Modelo de Advección-Difusión-Reacción
- ▶ **Brusselator1D**: Modelo de Reacción-Difusión
- ▶ **Brusselator2D**: Forma bidimensional

Tecnologías de paralelismo

OpenMP

- ▶ Interfaz de programación paralela
- ▶ **Arquitecturas multiprocesador** de memoria compartida
- ▶ Modelo de ejecución **fork-join**
- ▶ Surge en 1997
- ▶ Multithreading

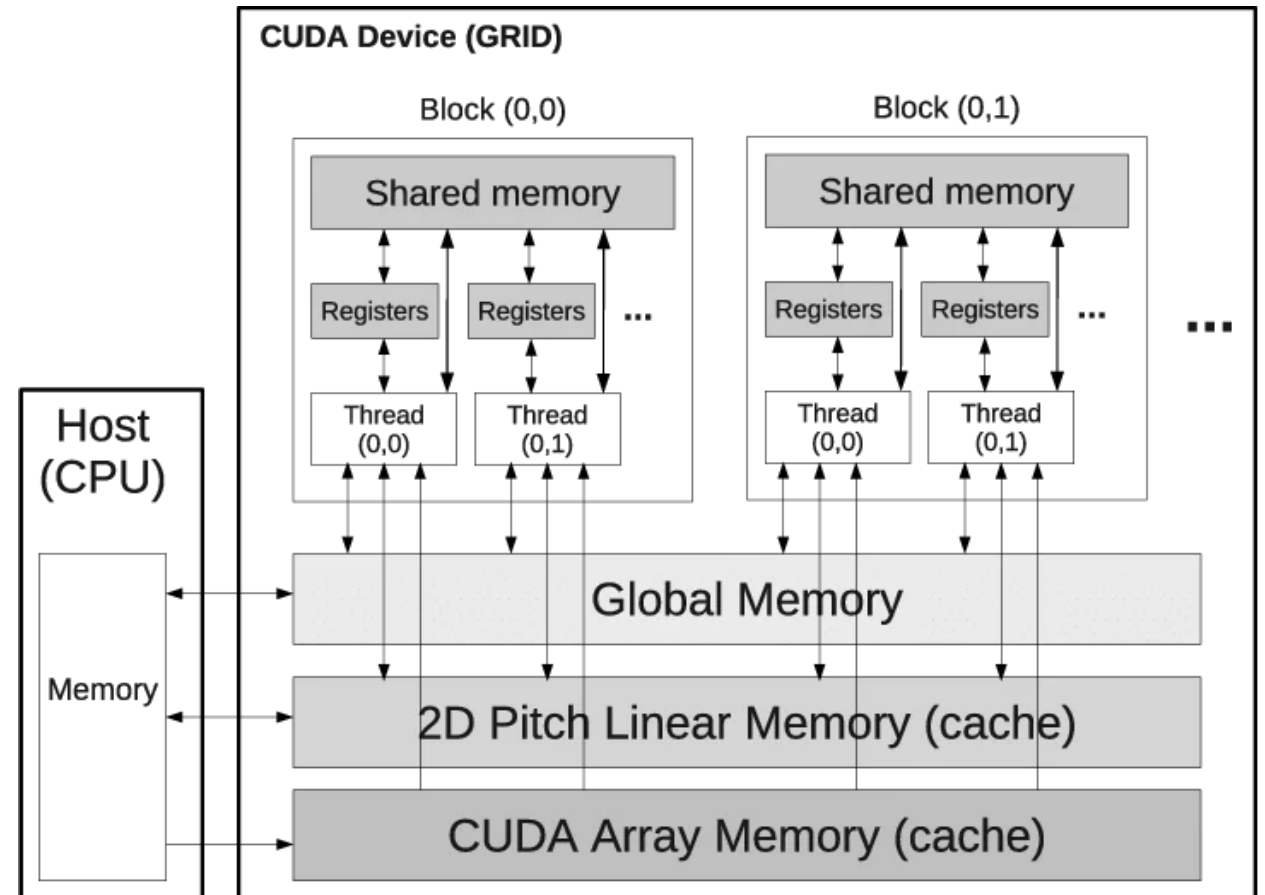


Modelo fork-join ([hpc-tutorials](http://hpc-tutorials.com))

CUDA

- ▶ Plataforma de computación paralela
- ▶ Ejecutar código en **GPUs**
- ▶ GPU -> Dispositivo de cómputo **masivamente paralelo**
- ▶ Nace en 2000
- ▶ Modelo **SIMT**, single instruction multiple threads
- ▶ **Warps** de 32 hilos en el SM
- ▶ **Host-Device**

14

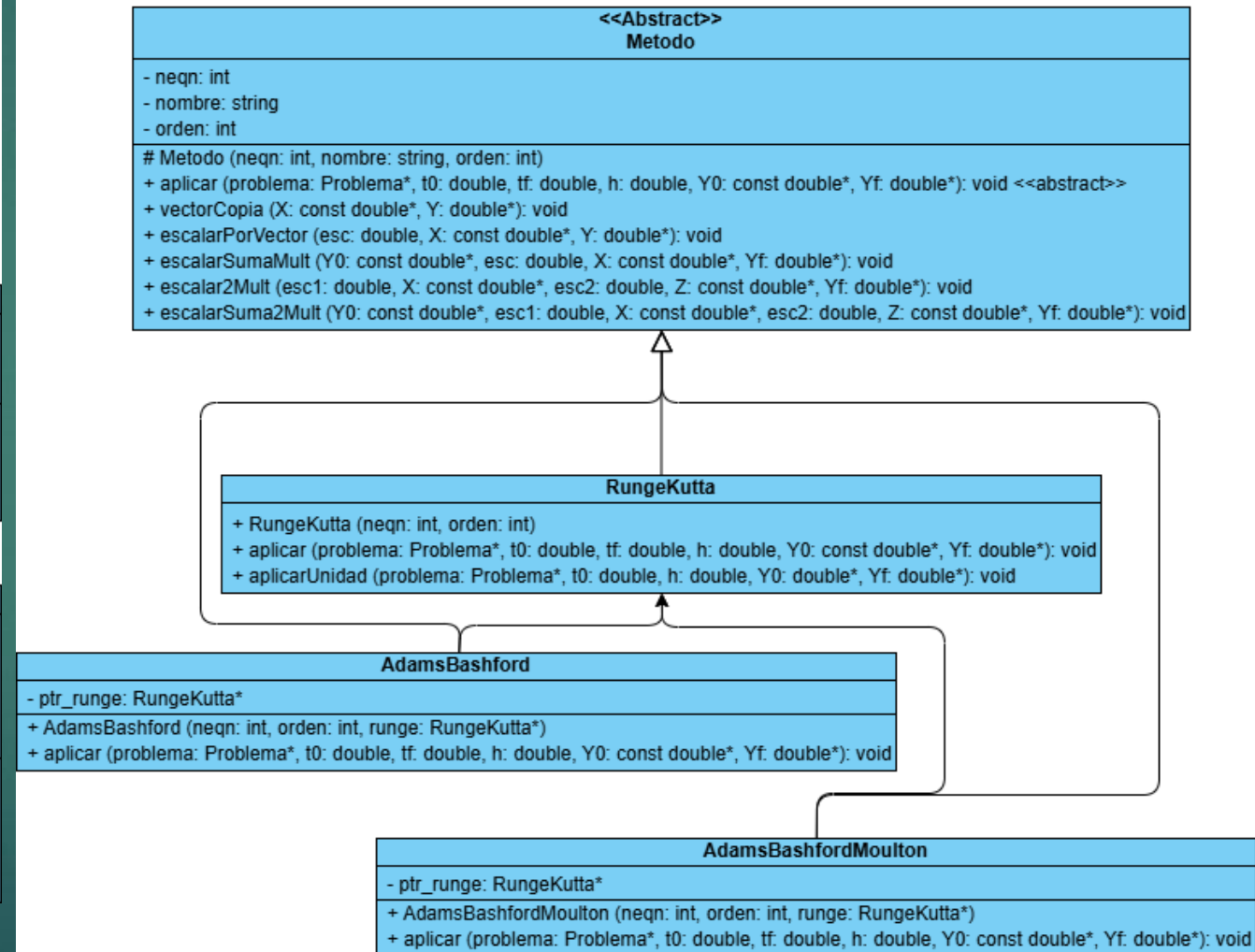
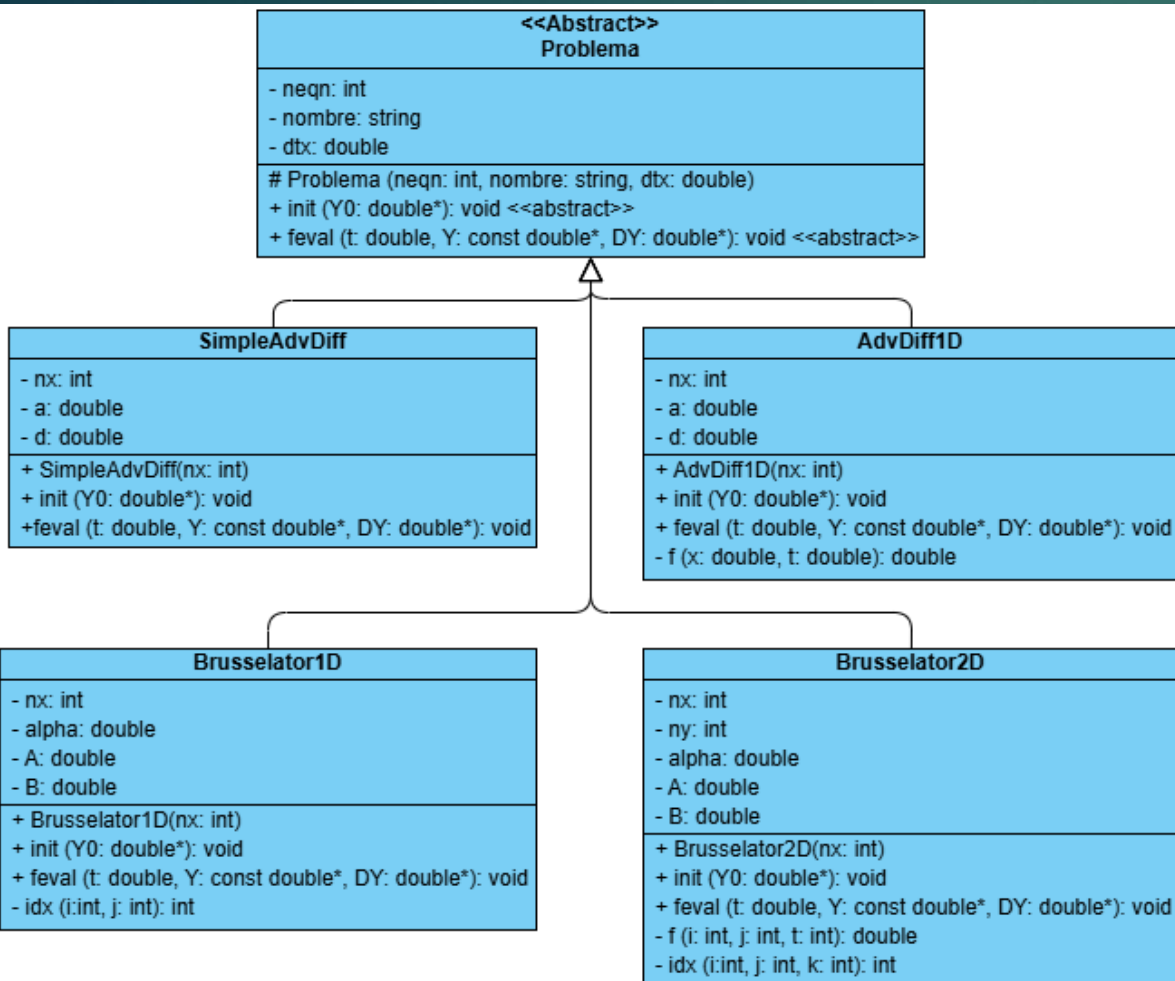


Arquitectura CUDA ([researchgate](https://www.researchgate.net/publication/220611144_CUDA_Architecture))

Implementación multihebra

Implementación secuencial

16



Paralelismo por operaciones

- ▶ Cada función paraleliza su propio trabajo internamente
- ▶ Creación de hilos, reparto del trabajo, sincronización y destrucción
- ▶ Reparto del trabajo función a función
- ▶ Modularidad
- ▶ Overhead

Algoritmo 5 Implementación de AB4 con Paralelismo por Operaciones

```

 $Y_{n0} \leftarrow Y_0$ 
for  $i=1; i < 4; ++i$  do
     $Y_i \leftarrow \text{arranqueRK4}(Y_{i-1})$ 
end for
for  $i=0; i < 4; ++i$  do
     $f_i \leftarrow \text{feval}(t_0 + i\Delta t, y_i)$ 
end for
for  $t_n=t_0+3\Delta t; t_n < t_f; t_n+=\Delta t$  do
     $y_4 \leftarrow \text{funcionesAuxiliares}(t_n, y_3, f_0, f_1, f_2, f_3)$ 
     $f_4 \leftarrow \text{feval}(t_n + \Delta t, y_4)$ 
     $\text{swap}(y_{n4}, y_{n3})$ 
    for  $i=1; i \leq 4; ++i$  do
         $\text{swap}(f_n, f_{n-1})$ 
    end for
end for
 $Y_f \leftarrow Y_{n3}$ 

```

Algoritmo 6 Implementación de Función Auxiliar vectorCopia

```

#pragma omp parallel for schedule(static)
for  $i=0; i < neqn; ++i$  do
     $Y[i] \leftarrow X[i]$ 
end for

```

Paralelismo por componentes

- Una gran región parallel
- **Único equipo de hilos** permanente
- **Menos barreras**
- Se trabaja sobre elementos individuales

Algoritmo 7 Implementación de AB4 con Paralelismo por Componentes

```

for j=1; j < 3; ++j do
     $y_j \leftarrow \text{arranqueRK4}(y_{j-1})$ 
end for
for j=0; j < 3; ++j do
     $f_j \leftarrow \text{feval}(t_0 + j\Delta t, y_j)$ 
end for
#pragma omp parallel
for  $t_n = t_0 + 3\Delta t$ ;  $t_n < t_f$ ;  $t_n += \Delta t$  do
    #pragma omp for schedule(static)
    for i=0; i < neqn; ++i do
         $y_4[i] = y_3[i] + \Delta t/24 * (55f_3[i] - 59f_2[i] + 37f_1[i] - 9f_0[i])$ 
    end for
    #pragma omp for schedule(static)
    for i=0; i < neqn; ++i do
         $f_4[i] \leftarrow \text{feval}_i(t_n + \Delta t, y_4, i)$ 
    end for
    #pragma omp single
    swap( $Y_{n4}$ ,  $Y_{n3}$ )
    for j=1; j ≤ 4; ++j do
        swap( $f_j$ ,  $f_{j-1}$ )
    end for
end for
 $Y_f \leftarrow Y_{n3}$ 

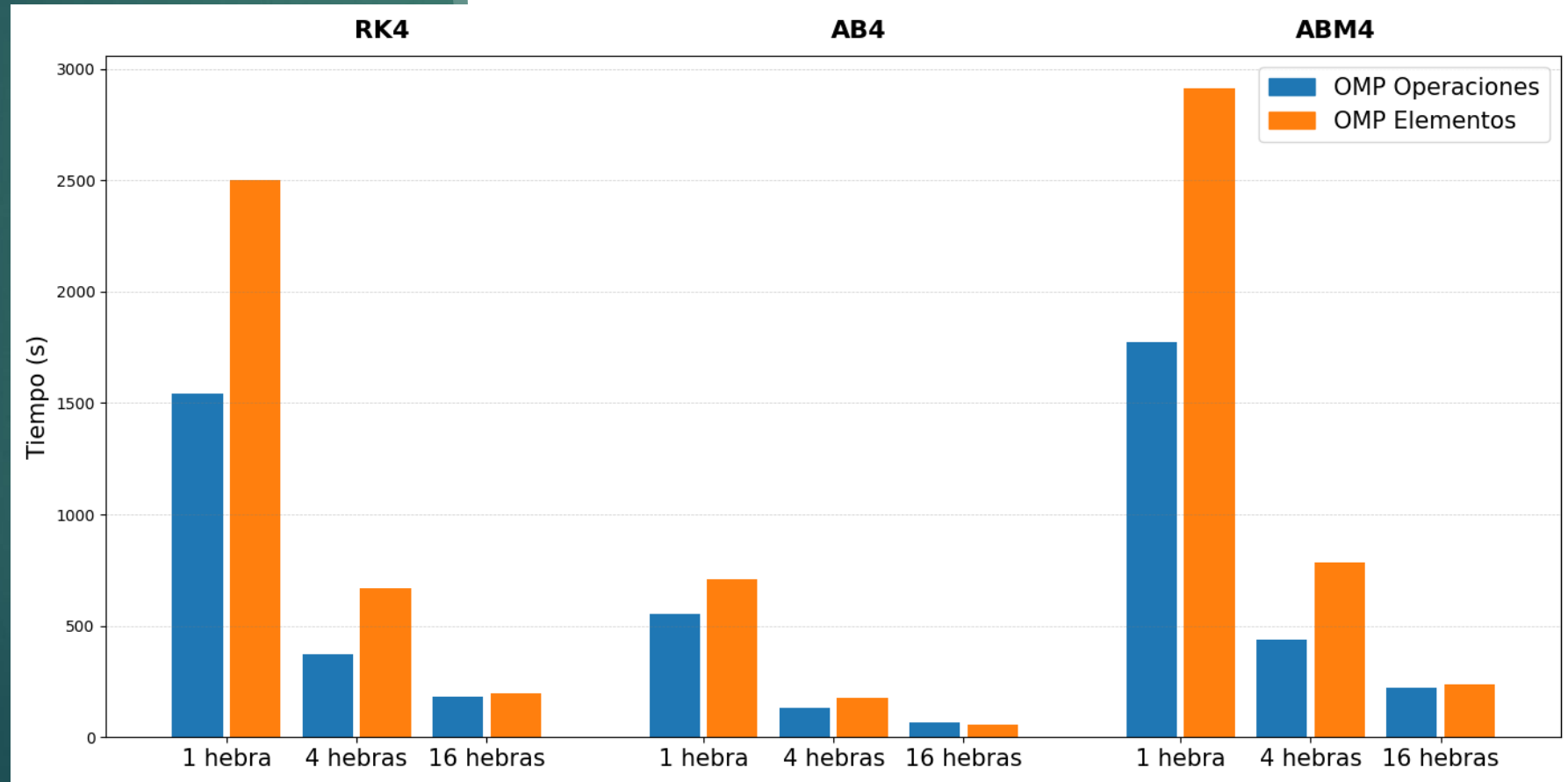
```

Resultados experimentales

19

Brusselator1D con
100K ecuaciones

- ¿Es mejor el paralelismo por componentes?
No.
- **Thread pooling**
- Hilos activos y dormidos



Comparativa de los tres métodos (elaboración propia)

Implementación para GPU

Planteamiento

- ▶ **Misma estructura:** clases abstractas padre (Problema y Método) y clases hijas que heredan de ellas
- ▶ Los kernels no son funciones miembro, hay que llamarlos
- ▶ Feval vacíos que **sólo llaman** a los kernels feval

- ▶ Funciones auxiliares



kernels auxiliares

- ▶ Memoria de constantes

Structs constantes



- ▶ **Modelo SIMT:**
todas las hebras ejecutan concurrentemente

Algoritmo 11 Kernel feval de AdvDiff1D

$thread \leftarrow blockDim.x \cdot blockIdx.x + threadIdx.x$

$ult \leftarrow neqn - 1$

if $thread < neqn$ **then**

▷ Check de hebras

$i_ant \leftarrow (thread == 0) ? ult : (thread - 1)$

▷ Vecinos y frontera

$i_pst \leftarrow (thread == ult) ? 0 : (thread + 1)$

$val_ant \leftarrow Y[i_ant]$

▷ Obtenemos valores

$valor \leftarrow Y[thread]$

$val_pst \leftarrow Y[i_pst]$

$DY[thread] \leftarrow$ Cálculo usando val_ant , $valor$, val_pst , $thread$, $t...$

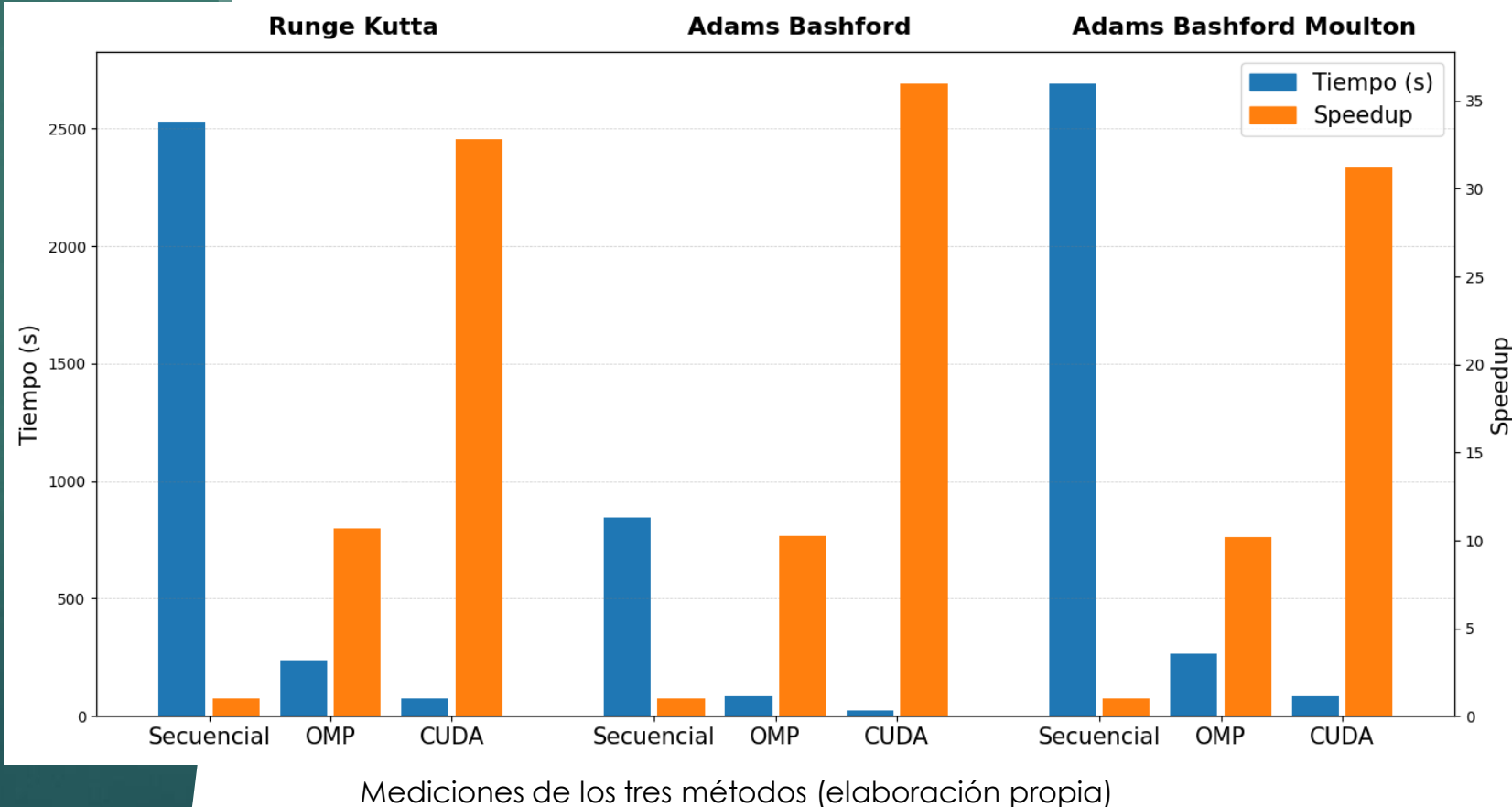
end if

Resultados experimentales

22

Brusselator2D con
125K ecuaciones

- **Ganancias en velocidad** similares en los 3 métodos
- **Speedup similares**, entre 30 y 35 en todas las instancias

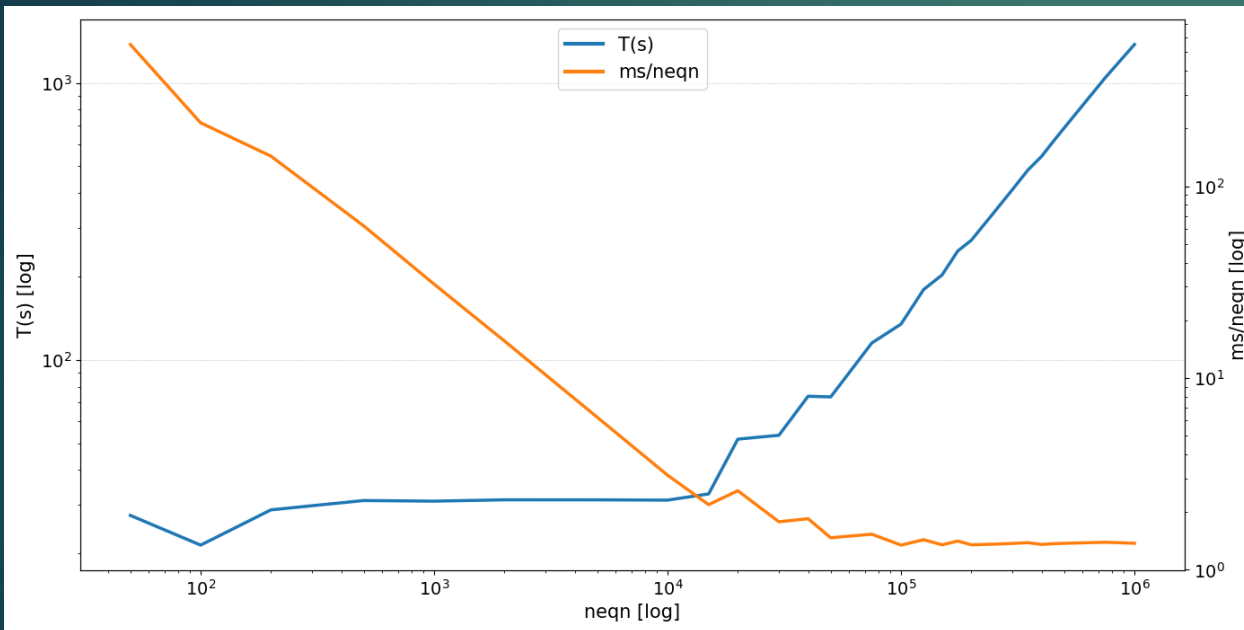


Resultados experimentales

23

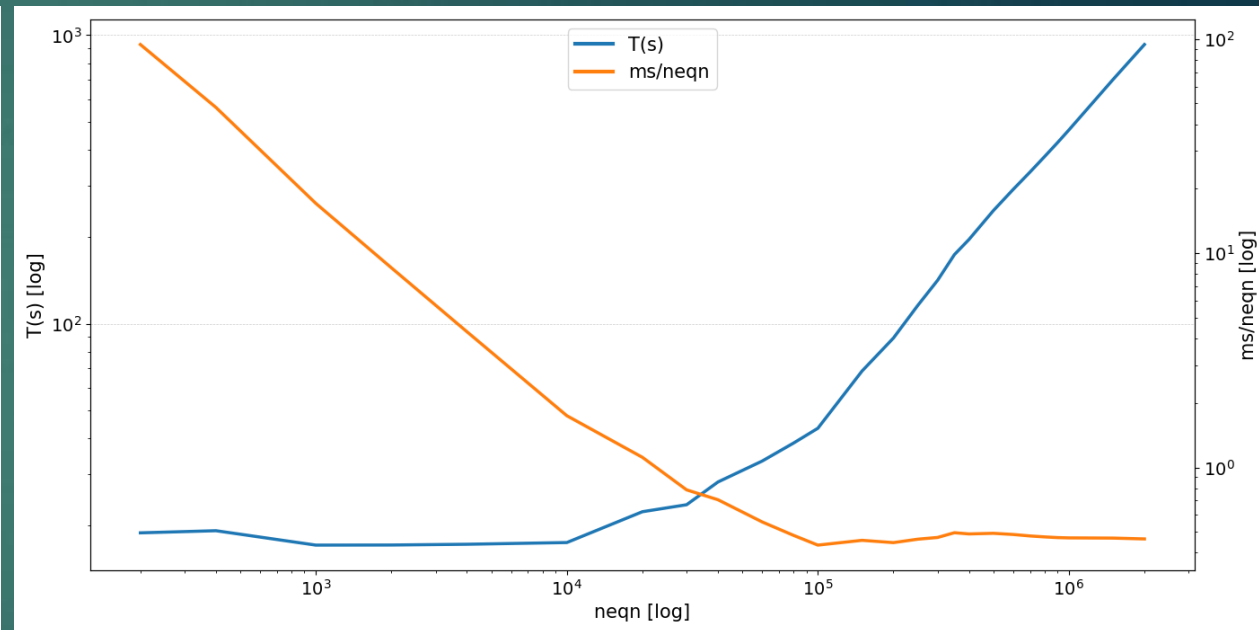
Diferentes tamaños, mismo método

Tiempo + tiempo por ecuación



Tiempo en función del tamaño (elaboración propia)

**RK4 resolviendo
AdvDiff1D**



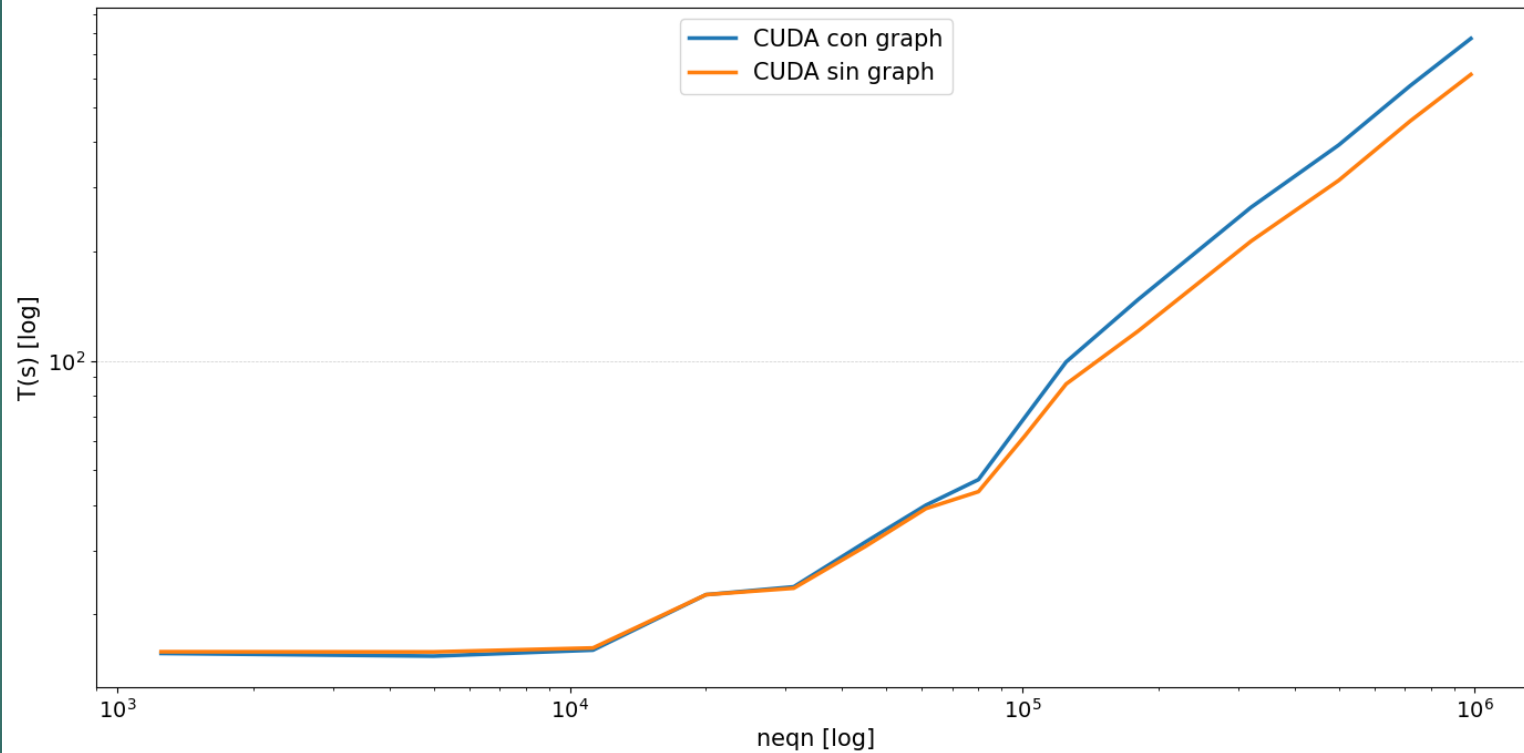
Tiempo en función del tamaño (elaboración propia)

**RK4 resolviendo
Brusselator1D**

CUDA Graphs

24

- Representación estática de un conjunto de operaciones y sus dependencias.
- Captura, análisis, optimización, instancia y ejecución



Comparativa con y sin CUDA Graph (elaboración propia)

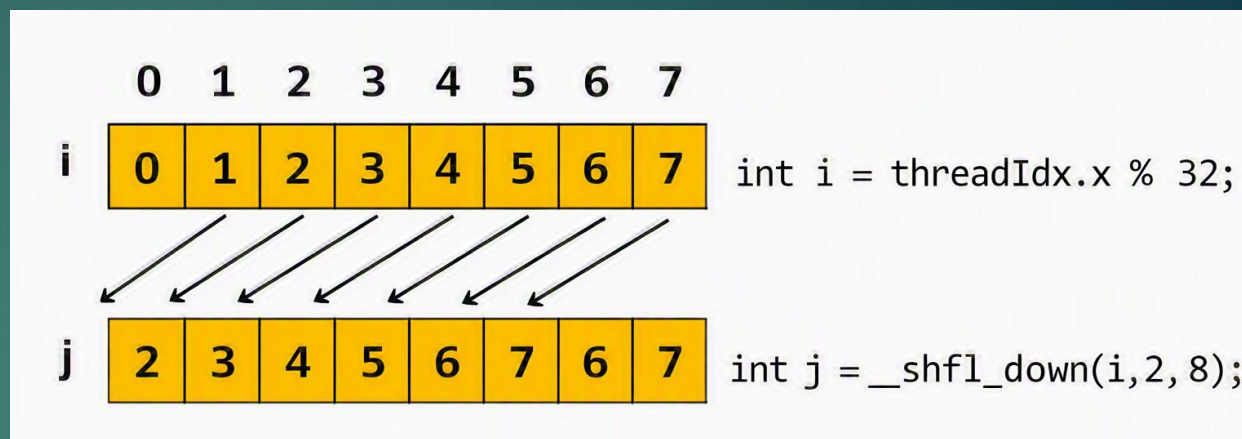
**ABM5 resolviendo
Brusselator2D**

Shuffles y grids multidimensionales

25

► Shuffles

- Minimizar acceso a memoria
- Instrucciones a nivel de warp
- Leer valor de un registro sin pasar por la memoria global

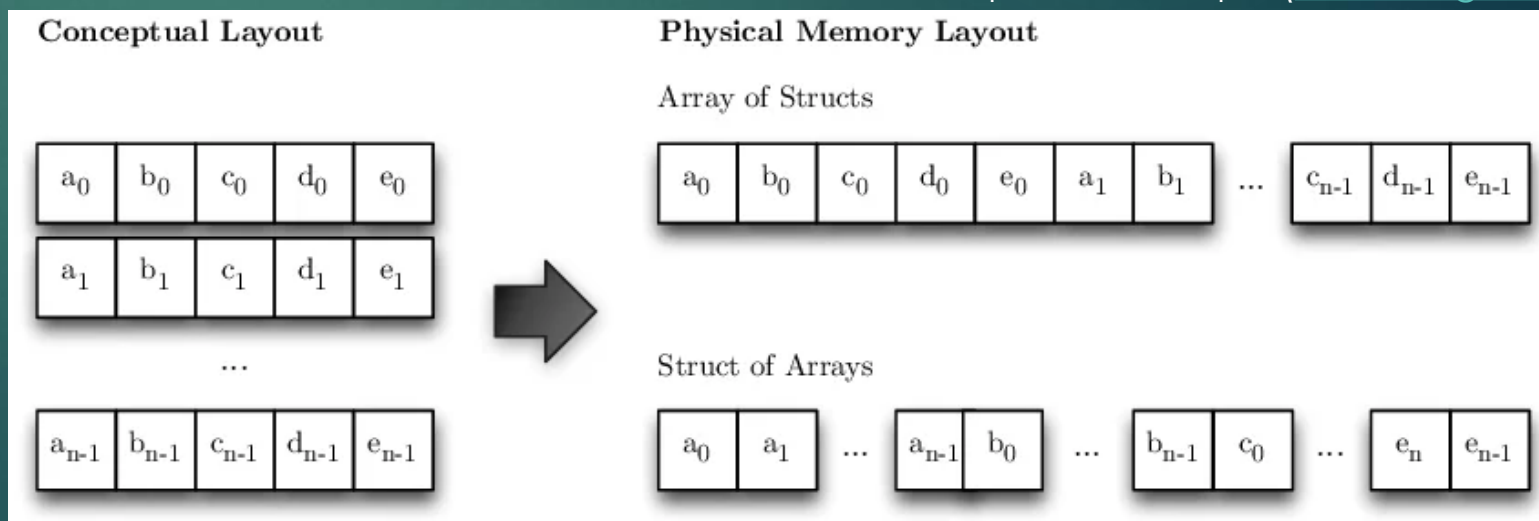


Uso de los shuffles ([nvidia](#))

Posibles estructuras para un bloque ([researchgate](#))

► Grids multidimensionales

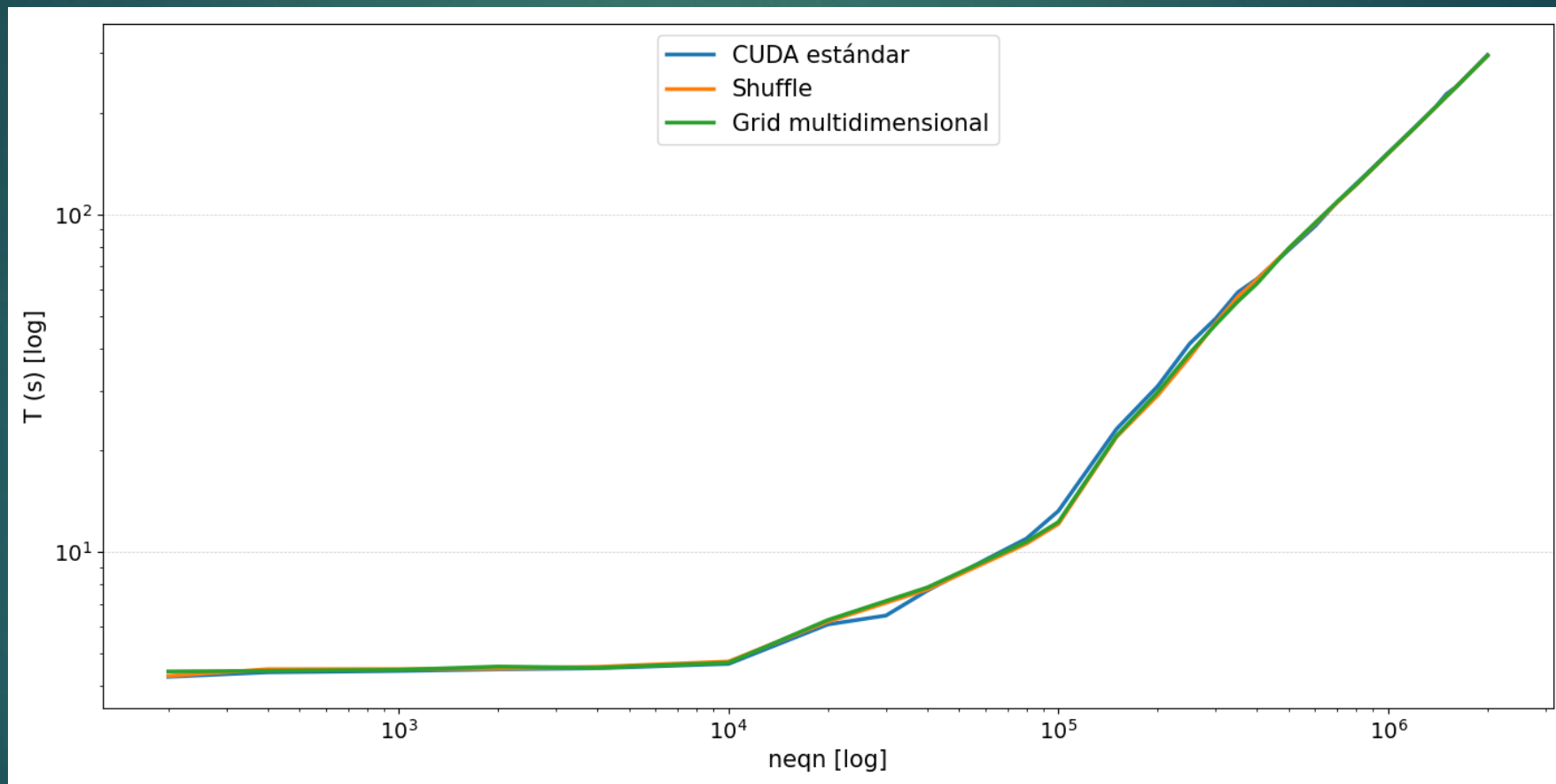
- Por defecto, bloques y grids de 1 dimensión
- Podemos especificar múltiples dimensiones
- Dividir cada componente en una dimensión distinta



Comparativa

26

AB4 resolviendo
Brusselator1D



Comparativa de ejecución con shuffles, y grid multidimensional (elaboración propia)

Conclusiones

Conclusiones

28

- ▶ Experiencia enriquecedora
- ▶ Base sólida
- ▶ Múltiples implementaciones, paradigmas, tecnologías...
- ▶ Análisis de los resultados
- ▶ Mejoras contundentes en GPU
- ▶ Herramienta esencial